# Billing Service Architecture

Document date: 2024-05-02 Classification: Internal

## Table of Contents

1. Overview ..... 2
2. Data Stores ..... 3
3. Dependencies ..... 4
4. Security and Authentication ..... 5
5. Scaling ..... 6
6. Open Questions ..... 7

# 1. Overview

The Billing Service (billing-service) is owned by the Finance Engineering team, with Noah Fischer as technical lead. It is implemented in Scala 2.13 on Akka HTTP and listens on port 8600. Secrets must never be committed; the pre-commit hook scans for high-entropy strings. Secrets must never be committed; the pre-commit hook scans for high-entropy strings. Load tests should be coordinated in the platform channel so they do not trip alerts.

The dashboards were updated to show saturation alongside latency and errors. Flaky integration tests in CI are tracked on the quality board; please do not just re-run them. The dashboards were updated to show saturation alongside latency and errors. Every service has a README that links to its dashboards and runbook.

# 2. Data Stores

Primary data store: PostgreSQL billing-db. Staging mirrors production topology but runs at roughly one tenth of the capacity. Flaky integration tests in CI are tracked on the quality board; please do not just re-run them.

# 3. Dependencies

Billing Service depends on: payment-service, order-service events, billing-db. Feature work is paused during incident follow-up weeks until action items are closed. Structured JSON logging is mandatory; free-text logs are dropped by the log pipeline.

# 4. Security and Authentication

Authentication mechanism: Billing Service uses mTLS between services plus OAuth2 client-credentials tokens. Structured JSON logging is mandatory; free-text logs are dropped by the log pipeline. Database schema migrations must be backward compatible for at least one release.

# 5. Scaling

Load tests should be coordinated in the platform channel so they do not trip alerts. Capacity planning for the holiday peak starts in September every year. Secrets must never be committed; the pre-commit hook scans for high-entropy strings. All services must expose Prometheus metrics on the /metrics path. Service-level objectives are reviewed quarterly with product owners.

Load tests should be coordinated in the platform channel so they do not trip alerts. Feature work is paused during incident follow-up weeks until action items are closed. Remember to tag pull requests with the owning team so reviews get routed correctly. Load tests should be coordinated in the platform channel so they do not trip alerts.

# 6. Open Questions

Database schema migrations must be backward compatible for at least one release. Flaky integration tests in CI are tracked on the quality board; please do not just re-run them. The dashboards were updated to show saturation alongside latency and errors.